

Topics :

- Recursion
- Pattern Matching
- Guards
- Cases

1

1

Definition

- The term recursion means **repetition**
- A recursive function therefore **defines the function in terms of itself**
- The **function can invoke itself** only when a certain condition is met
- **Example 1: Factorial**
 - $n!$ or $fac\ n$, denotes the factorial of n
 - $n! = fac\ n = n \times (n-1) \times (n-2) \times \dots \times 2 \times 1$
- Specification for $fac\ n$

$$fac(n) = \begin{cases} 1, & \text{if } n = 0 \\ n * fac(n - 1), & \text{if } n > 0 \end{cases}$$

2

2

Structure of recursive definitions

❖ Recursive definitions have the following structure:

- **Base case** – defines the function for the **first input values**, it **terminates** the recursion
- **Recursive step** – defines the function for **other input values** in terms of itself, creating a **loop**
- Without the base case, the function may remain in an indefinite loop

3

3

- The recursive declaration has **two definitions**: one for the base case and one for the recursive case
- The **order of the declarations is important** in Haskell
- The compiler selects the declaration to use **starting at the top**
- Then pick the one that matches
- **Always list multiple function definitions starting with the most specific and proceeding to the most general**

4

4

- Basic rules to remember when doing recursion:
 - Identify the end goal(s)
 - Determine what happens when a goal is reached
 - List all alternate possibilities
 - Determine your “rinse and repeat” process
 - Ensure that each alternative moves you toward the goal

5

5

Example 1: factorial

❖ Example : factorial function

- Factorial function takes a positive integer n and finds all positive integers less than or equal to n and multiplies them all together
- e.g.,

factorial 6 = 6!	=	6*5*4*3*2*1
factorial 5 = 5!	=	5*4*3*2*1
factorial 4 = 4!	=	4*3*2*1
factorial 3 = 3!	=	3*2*1
factorial 2 = 2!	=	2*1
factorial 1 = 1!	=	1
factorial 0 = 0!	=	1

1 <= an exception, factorial is only for integers

=> a factorial of a number is just that number multiplied by the factorial of a number one less than it

6

6

❖ Example : factorial function

- A factorial of 0 is 1
- A factorial of a number is just that number multiplied by the factorial of a number one less than it
 - **Base case:** $\text{fac } 0 = 1$
 - **Recursive step:** $\text{fac } n = n * \text{fac } (n - 1)$

$$\begin{aligned}
 \text{e.g. fac } 4 &= 4 * \text{fac}(3) \\
 &= 4 * 3 * \text{fac}(2) \\
 &= 4 * 3 * 2 * \text{fac}(1) \\
 &= 4 * 3 * 2 * 1 * \text{fac}(0) \\
 &= 4 * 3 * 2 * 1 * 1 = 24
 \end{aligned}$$

7

7

Dry running recursive function: *fac 5*

9

9

- The factorial function can be defined in Haskell at the prompt

```
Prelude>:{
Prelude|let factorial 0 = 1  -- the base case
Prelude|    factorial n  = n*factorial(n-1) --the recursive step
Prelude|}:
Prelude>factorial 6
720
Prelude>
```

10

10

- As an alternative, the function can be defined as a **one liner**, separating the lines by a **semi colon**

```
Prelude>let factorial 0 = 1; factorial n  = n*factorial(n-1)
Prelude>factorial 6
720
Prelude>
```

11

11

Fibonacci sequence F_n

▪ Specification for fib (n)

$$\text{fib}(n) = \begin{cases} 0, & n = 0 \\ 1, & n = 1 \\ \text{fib}(n-1) + \text{fib}(n-2), & \text{otherwise} \end{cases}$$

14

14

Fibonacci sequence F_n

▪ Base case:

○ Fibonacci of 0 is 0 and Fibonacci of 1 is 1

➤ $F_0 = 0, F_1 = 1$

▪ Recursive step

○ This computes the nth Fibonacci number in the series

○ The fibonacci of n is adding the fibonacci of n-1 and the fibonacci of n-2

➤ $F_n = F_{(n-1)} + F_{(n-2)}$

▪ Any integer in the sequence is the sum of the previous 2 integers

15

15

Exercise - Dry run: F_5

16

16

- The fibonacci sequence can be defined in Haskell at the prompt

```
Prelude>:{
Prelude|let fib 0 = 0           -- first integer
Prelude|    fib 1 = 1           -- second integer
Prelude|    fib n = fib(n-1) + fib(n-2) --recursive step
Prelude|:}
```

17

17

Making Choices

- In Haskell, recursive functions can be expressed using:
 - If then else
 - Pattern matching
 - Guards
 - Case statement

19

19

If-then-else

- If then else provides a way of using **conditional expressions**
- Used to define functions whose **results depend on arguments**
- In Haskell **if is not a statement, but an expression**, meaning that **it returns a value**
- So, **if always has a type**, always **has an else case**, which must return a value of the same type
- Can use **nested if statements**
 - Not easy to read
 - Expand quickly in size and complexity as number of cases increase

20

20

fac n Using If-then-else

- Specification for *fac n*

$$\text{fac}(n) = \begin{cases} 1, & \text{if } n = 0 \\ n * \text{fac}(n-1), & \text{if } n > 0 \end{cases}$$

- Using if then else

```
fac :: Integer -> Integer -- takes an integer, returns integer
fac n =
    if n==0 then 1
    else n*fact(n-1)
```

- Recall that **indentation is important** in Haskell

21

21

fibonacci n Using If-then-else

- Specification for *fib n*

$$\text{fib}(n) = \begin{cases} 0, & n = 0 \\ 1, & n = 1 \\ \text{fib}(n-1) + \text{fib}(n-2), & \text{otherwise} \end{cases}$$

- Using if then else

```
Prelude> :{
Prelude| fib n =
Prelude|     if n==0 then 0
Prelude|     else if n==1 then 1
Prelude|     else fib(n-1) + fib(n-2)
Prelude| :}
```

22

22

Pattern Matching

- Pattern matching is a way of **expressing choices** in Haskell
- Expressions are used to **choose between results of the same type**
- It provides **multiple syntactic expressions** called *patterns*, typically **based on a data constructor/variant**, and the **compiler will match the most appropriate one**
- Pattern matching **is similar to if..then..else** statements, but provides code that is
 - neat
 - readable
 - simple

23

23

Pattern Matching

- Pattern matching is **sequential** in Haskell
 - Arguments are **matched with the patterns in sequence**
- The **first equation which applies is used**
- Always include a **catch all pattern** so that the program does not crash
- The wildcard sign **_ is used as a catch all to match anything**
- Leaving it out typically **results in a warning message** “Pattern matches are **non exhaustive**”

24

24

Pattern Matching

❖ Syntax for pattern matching

```
function_name::input_types->output_type --define the function
function_name Pattern1 = firstvalue_to_return
function_name Pattern2 = secondvalue_to_return
function_name Pattern3 = thirdvalue_to_return
```

➤ You can have **any number of cases or patterns**

- We give the **name of the pattern** and after this **value to be matching**, followed by the **value that we want to return**
- If the first pattern is matched, the first result is chosen
- If the second pattern matches, the second result is chosen
- Etc..

25

25

Pattern Matching

❖ A pattern can be a

- **literal** value – an argument matches the pattern if it is equal to the value
- **variable** – any argument value will match
- **wildcard** – the catch all sign `_`
- **tuple pattern** – the argument must be of the form `(v1, v2,...,vn)`
- **constructor** applied to a number pattern

26

26

Example 1: `negate` function

- A function called `negate` to negate the boolean variables `True` and `False`

i.e. `negate True` should return `False`

`negate False` should return `True`

- Function definition?
- Implementation?

27

27

- Using pattern matching for the `negate` function

```
negate::Bool->Bool    --function definition
negate True = False   --first pattern
negate False = True   --second pattern
```

- First line is the function definition
- The second line is the first pattern to match for `True`, if it is matched, then the result is `False`
- In the next line, the second pattern to negate `False`, which gives `True` as the result if matched
- There are only two patterns

28

28

Example 2: square function

- A function called `square` to squares integers. Take the square of 0 as 1

i.e. `square 0` should return 1

`square y` should return $y*y$

- Function definition?
- Implementation?

29

29

```
square::Int->Int -- function definition
square 0 = 1     --first pattern
square y = y*y   -- second pattern
```

- In line 1, we enter the square function, which squares an integer and returns an integer
- In line 2, we create our first pattern to match for 0
- In line 3, we create our second pattern to square any other supplied value besides 0

30

30

fac n using pattern matching

```
fac :: Integer -> Integer -- takes an integer, returns integer
fac 0 = 1 -- the pattern is the literal value 0
fac n = n*fac(n-1) -- pattern is a variable n matching anything
```

- There is a number of conditional equations
- Each equation has a left-hand side in which the function is applied to all patterns

31

31

Fibonacci sequence using pattern matching

```
fib :: Integer -> Integer
```

32

32

Guards

- Guards are also used to give **choices**
- Gives **alternatives** in definitions of functions
- A guard is a **boolean expression**
 - tests if a property of some value is **true or false**
 - used to express different cases when defining functions
 - a case of a pattern is taken based upon some condition
- Guards are **sequential**
 - Options are checked in the way listed
 - Ordering is important

34

34

Guards

- Start with the function **signature, declaration, and variables**
 - Do not use the = sign
- Go to a new line and **use the pipe | character to set up the various options**
- Each line with a | represents a **different option**
- There can be **multiple guards**
- The options are represented **using boolean expressions**: statements that will either be True or False
- The **last option should catch all possible options**, using the `otherwise` keyword
- There can be **two or more guards**

35

35

fac n using Guards

```
fac :: Integer -> integer  -- takes an integer, returns integer
fac n
  | n == 0      = 1
  | otherwise   = n * fac(n - 1)
```

- Options are checked in order that they are listed
- Careful ordering

36

36

Fibonacci using guards

```
fib :: Integer -> integer  -- takes an integer, returns integer
```

37

37

Cases

- When using guards, each time we want to check what the value of the variable, we need to rewrite `variable ==`
- Luckily, Haskell has another way of making decisions: **Cases**
- Syntax is
 - `case expression of`
 - `where expression is some expressions that returns some value`
- Then `->` is used to mean 'should return'
- The wildcard is represented as `_` to catch all

39

39

Cases

- `case expression of`
- Expression can be a
 - single variable
 - tuple
 - calculation
 - function applied to variables

40

40

Example 1 sumNumbers n

❖ `sumNumbers n` is a recursive function that computes sum of all positive whole numbers up to and including `n`

▪ **Specification of `sumNumbers n`**

$$\text{sumNumbers } (n) = \begin{cases} 1, & \text{if } n = 1 \\ n + \text{sumNumbers } (n - 1), & \text{if } n \geq 1 \end{cases}$$

▪ **Base case:** `sumNumbers (1) = 1`

▪ **Recursive Step:** `sumNumbers (n) = n + sumNumbers (n-1)`

41

41

sumNumbers n Using pattern matching

```
sumNumbers :: Integer -> Integer
```

```
sumNumbers n
```

```
sumNumbers 1 = 1
```

```
sumNumbers n = n + sumNumbers (n - 1)
```

42

42

sumNumbers n Using Guards

```
sumNumbers :: Integer -> Integer
sumNumbers n
  | n == 1      = 1
  | otherwise   = n + sumNumbers (n - 1)
```

43

43

Handling Errors

- **Factorial** is only defined for positive integers
- *fac* (-1) should return short informative error messages
- **Example:** *fac* (- 4)
 - ⇒ (- 4) x *fac* (- 5)
 - ⇒ (- 4) x (- 5) x *fac* (- 6)
 - ⇒
 - ⇒ **This is an infinite loop**

44

44

Improved factorial function with error handling

`fac :: Integer -> Integer`

`fac n`

`| n < 0 = error "fac only defined for positive integers"`

`| n >= 0 && n <= 1 = 1`

`| otherwise = n x fac ( n - 1 )`

45

45

Exercise: Recursive definition of rangeSum

- **rangeSum** - takes two integers and returns the sum of all integers from the first to the second, inclusive.
- **E.g.** `rangeSum 3 5`
12
- 1. Provide a specification for this function, take care of `rangeSum b a`, where $b > a$. example `rangeSum 9 6`
- 2. Dry run `rangeSum 3 5` using the notation introduction in previous slide.
- 3. Provide a Haskell implementation for `rangeSum`.

46

46

1. Specification of *rangeSum*

$$\text{rangeSum}(m, n) = \left\{ \right.$$

47

47

2. Dry running: *rangeSum* 3 5

48

48

3. Provide implementation for: *rangeSum*

49

49

Exercise – Take home

- Give a recursive definition of a function **sumFac** defined as follows:

$\Rightarrow \text{sumFac} :: \text{Integer} \rightarrow \text{Integer}$

$\Rightarrow \text{sumFac } n = \text{fac } 0 + \text{fac } 1 + \dots + \text{fac } (n - 1) + \text{fac } (n)$

54

54

